Vu-Le *et al. Applied Network Science* (2026) 11:53
https://doi.org/10.1007/s41109-026-00809-z

Applied Network Science

RESEARCH Open Access

Improving community detection with CVC, a new cluster ensemble method

The-Anh Vu-Le[1], Minhyuk Park[1], Ian Chen[1], João Alfredo Cardoso Lamy[2], Tomás Alessi[2], Elfarouk Harb[1], George Chacko[1] and Tandy Warnow[1*]

*Correspondence:
Tandy Warnow
warnow@illinois.edu
[1]Siebel School of Computing and Data Science, University of Illinois Urbana-Champaign, 201 N. Goodwin Avenue, Urbana, IL 61801, USA
[2]Insper Institute, Rua Quatá, 300, São Paulo, SP 04546-042, Brazil

Abstract
Community detection is an unsupervised learning problem with many applications. In this study, we present Constrained Voting Consensus (CVC), a new ensemble clustering method that is designed to combine multiple clusterings, some of which may only cover a small portion of the network. We demonstrate that combining several dense subgraph clustering methods with standard methods within this ensemble approach produces a more accurate clustering than its constituent clustering methods. We also compare CVC to other ways of combining sets of clusterings, including the median consensus method, and show that CVC achieves higher accuracy and has better robustness. Furthermore, CVC can scale to very large networks with millions of vertices.

Keywords Community detection, Dense subgraphs, Cluster ensemble

Introduction

Community detection, also known as graph clustering, is the task of partitioning the vertices of a network into disjoint groups so that each group has features indicative of being a community. While there are differing criteria for what constitutes a "good community", typical features that are sought are high edge-density, good separation from the rest of the network, and being well-connected (Yang and Leskovec 2013).

There are many methods used for community detection, varying in approach, theoretical guarantees, and empirical performance. Among the well-established methods are modularity optimization (Newman 2004), often approached using the Blondel et al. (2008) software, and optimizing under the Constant Potts Model (Traag et al. 2011), enabled by the Leiden (Traag et al. 2019) software. These are NP-hard optimization problems, and so the methods used are based on heuristic searches that are not guaranteed to find optimal solutions. Other approaches have been developed that operate in a greedy fashion: find a high quality subgraph, label it as a "community", then delete it from the network and repeat, until there are no more high quality subgraphs. The Iterative k-core (IKC) method, presented in Wedell et al. (2022), is an example of this type of method; this approach finds the largest k so that the network has a k-core (i.e.,

© The Author(s) 2026. **Open Access** This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

a subgraph where every node is adjacent to at least k other nodes in the subgraph), extracts the relevant k-core and labels it as a community, and then repeats, until there are no k-cores with $k \geq B$, where B is a user-specified parameter. In Wedell et al. (2022), this method was tested with $B = 10$.

In prior work (Vu-Le et al. 2026), we explored community detection based on iterative methods that operate by finding and extracting edge-dense subgraphs. Among all the methods we explored, the most accurate was DSC-Flow-Iter (a method based on a method for dense subgraph extraction in Harb et al. (2025)). This DSC-Flow-Iter method had high precision but low recall, and had low node coverage, leaving much of each network unclustered. Our study then showed a new ensemble technique and used it to combine DSC-Flow-Iter with two other community detection methods–Leiden optimizing modularity and Leiden-optimizing the Constant Potts Model (CPM). The study in Vu-Le et al. (2026) showed that the combination of these three methods was more accurate than its constituent parts. However, the ensemble technique we provided was unable to scale to very large networks, where it reported OOM errors.

Here we extend the study presented in Vu-Le et al. (2026) in several ways. First, we provide an exploration of a larger number of community detection methods that are based on finding and extracting high-quality subgraphs: several that are based on edge density and one that is based on triangles in the subgraphs. Second, we explore the design space of ensemble methods, focusing in depth on the selection of community detection methods to include in the ensemble, as well as how to combine the clusterings they produce; this part includes a careful ablation study. Third, we provide results for a re-engineered version of our clustering ensemble pipelines that has better memory management than the version studied in Vu-Le et al. (2026), and so can successfully be run on even the CEN, the largest network we studied, with $\sim$ 14 million nodes. Fourth, we compare our pipelines to the Median Consensus (Hussain et al. 2025), a prior clustering consensus method that can take arbitrary input clusterings and has been shown to scale to very large networks.

The rest of the paper is organized as follows. We begin with background material, and then describe materials and methods, including the new iterative community detection methods we explore and the new ensemble methods we developed. We follow with results of the experiments. We finish with conclusions, where we also describe future work. Some additional results are provided in the supplementary materials.

Background

Overview of main clustering methods

We study a problem where the input is a graph G (also called a network) with edges and vertices, and the goal is to partition the nodes into disjoint sets so that each set exhibits good community structure.

Many methods have been developed for community detection, including methods that can be categorized into optimization-based approaches and iterative extraction techniques. Optimization-based methods reframe the clustering problem as finding the optimal partition of the vertex set with respect to a specific objective function. Since many optimization problems are NP-hard, heuristics are typically used to find good, possibly globally optimal, solutions. For instance, the Traag et al. (2019) algorithm is a heuristic search that can be used for either modularity (Leiden-Mod) or the Constant Potts Model

(Leiden-CPM), while Rosvall et al. (2009) aims to minimize the map equation. Similarly, the graph-tool library (Peixoto 2014) focuses on optimizing the posterior probability under various stochastic blockmodels (SBMs), including degree-corrected and hierarchical variants.

Modularity clustering is one of the most popular approaches, but has been shown to be subject to a "resolution limit" (Fortunato and Barthélemy 2007), which in essence means that it may fail to recover communities that are too small. Other studies (e.g., Leskovec et al. 2010; Vu-Le et al. 2026) have shown that optimal modularity clustering tends to produce very large sparse clusters. The Constant Potts Model (CPM), introduced in Traag et al. (2011), was developed to address the limitations of modularity optimization, and allows the user to specify a resolution parameter $\gamma \in [0, 1]$, and then guarantees that the optimal $CPM(\gamma)$ clustering will be "well-connected", i.e., the edge cuts for any cluster in the optimal clustering will never be below a size that depends on γ. In addition, optimal CPM clusterings were proven to be "resolution-limit-free" (Traag et al. 2011). These results are based on optimal solutions to modularity- or CPM-based clusterings. Since these are NP-hard optimization problems, heuristics are used to find good (even if not optimal) solutions, with Blondel et al. (2008) and Traag (2019) popular software for modularity and CPM-optimization, respectively. Less-studied is iterative extraction of high quality communities, which we discuss in a later section.

Post-processing techniques

In conjunction with designing methods for community detection, methods have also been developed to post-process clusterings in order to improve accuracy. For example, the Connectivity Modifier (Park et al. 2024a) was developed to address the observation that many clustering methods produced clusters that had small edge cuts, so that they did not pass a user-specified threshold for being "well-connected". The Connectivity Modifier (CM) operates on an input clustering by finding clusters that fail the well-connectedness test, removing a small edge cut and reclustering, until all clusters pass the test. This method improved accuracy for several standard clustering methods, such as Leiden optimizing under the Constant Potts Model or optimizing Modularity. A simpler version of this technique that applies the same strategy but does not recluster in each iteration was studied in Vu-Le et al. (2026), and is referred to as Well-Connected Clusters (WCC). WCC also improves accuracy. The simplest version of this technique is Connected Components (CC), which simply replaces any cluster that has two or more components by its connected components. The CC method is appropriate for methods such as Louvain, Infomap, and stochastic blockmodels, that have been shown to sometimes produce disconnected clusters (Traag et al. 2019; Park et al. 2024b; Vu-Le et al. 2026).

Iterative subgraph extraction

A natural algorithmic approach to community detection is to find high-quality subgraphs, label them as communities, then delete them and repeat, until there are no high-quality subgraphs remaining. Despite the naturalness of this approach, there has been relatively little exploration of the technique in the literature. We are aware of three studies that have examined this approach. One is based on repeatedly finding and extracting a connected subgraph that has the largest coreness, where the coreness of a subgraph

is the maximum k so that each node is adjacent to at least k nodes in the subgraph. The Iterative k-core (IKC) (Wedell et al. 2022) uses this technique, with the option to only include subgraphs that have positive modularity. Another similar iterative subgraph extraction method is RTRex (Basu et al. 2024), which seeks the subgraph that has the largest density of triangles (i.e., 3-cliques). Finally, in Vu-Le et al. (2026), we explored methods that iteratively find and delete dense subgraphs, as we now describe.

The standard definition of the density of a subgraph X is the number of edges in X divided by the number of nodes in X. The Dense Subgraph Discovery problem, which finds the subgraph with maximum density, is solvable in polynomial time (Goldberg 1984; Gallo et al. 1989). Furthermore, a range of approximation algorithms exists, from the 1/2-optimal greedy algorithm by Charikar (2000) to the nearly optimal Greedy++ algorithm by Boob et al. (2020). These approximation techniques often reformulate the discovery problem as a linear program, subsequently giving rise to a broader optimization challenge known as the Dense Subgraph Decomposition problem. This problem extends the goal from finding a single densest subgraph to partitioning the entire graph based on density considerations.

Formally, the Dense Subgraph Decomposition problem aims to partition the vertex set of an input graph $G = (V, E)$ into a sequence of subsets $S_1, S_2, \ldots$ through a recursive optimization process. The first subset, S_1, is identified as the set of nodes that induces the densest subgraph of G. In subsequent steps, the procedure identifies S_i as:

$$S_i = \underset{S \subseteq V \setminus U_{i-1}}{\arg\max} \frac{|E(S)| + |E(S, U_{i-1})|}{|S|}$$

where U_{i-1} denotes the union of all previously identified subsets (i.e., $U_{i-1} = S_1 \cup \cdots \cup S_{i-1}$), $E(S)$ contains the edges between nodes of S, and $E(S, U_{i-1})$ contains the edges between nodes of S and nodes of U_{i-1}. Since the standard density is $|E(S)|/|S|$, this formulation essentially also searches for the "densest" remaining subgraph but with a modified notion of density that accounts for the connections to the already extracted regions, similar to a peeling process with the densest subgraph at the core.

Various algorithms have been proposed to solve the dense subgraph decomposition problem, many of which are grounded in Charikar's linear programming formulation. Techniques such as Greedy++ employ the multiplicative weight update (MWU) framework, while others extend the problem to quadratic programming solved via the Frank-Wolfe algorithm (Danisch et al. 2017). More recently, Harb et al. introduced two distinct solvers: FISTA, which solves a convex relaxation of the problem using the Fast Iterative Shrinkage-Thresholding Algorithm (Harb et al. 2022), and Flow, which utilizes a max-flow formulation solved via a push-relabel algorithm (Harb et al. 2025). A key feature of both methods is the intermediate assignment of a value to each vertex that approximates the modified density of the subset to which the vertex belongs within the decomposition hierarchy.

In Vu-Le et al. (2026), we adapted the FISTA and Flow algorithms for community detection through two methodologies to propose four dense subgraph clustering (DSC) techniques. The first approach, which we termed DSC-FISTA(int) and DSC-Flow, directly groups nodes based on their assigned values, essentially using the approximated decomposition as our clustering. Since FISTA values are inaccurate approximations, we apply rounding before grouping; on the other hand, Flow values are better

approximations that can be used as is. Furthermore, if a resulting cluster contains disconnected components, we separate each of them into a community. The second approach aligns more strictly with the iterative extraction paradigm: we utilize the algorithms as subroutines to identify only the single densest subgraph (the first subset in the decomposition), remove it, and repeat the process on the remaining graph. For the FISTA-based variant, we employed the fractional peeling technique to approximate the densest subgraph, while for the Flow-based variant, we selected the group of nodes with the highest assigned value. These iterative variants were named DSC-FISTA-Iter and DSC-Flow-Iter. Note that although an iterative version of DSC-FISTA(int) is an intuitive extension, it was not evaluated in Vu-Le et al. (2026).

Cluster ensembles

The fundamental objective of the cluster ensemble (or consensus clustering) problem is to aggregate multiple base clusterings into a single, unified clustering that exhibits superior stability and accuracy. Within the specific domain of network community detection, several consensus architectures have been established to solve this problem, surveyed in Hussain et al. (2025). This survey identified two major types of consensus methods that provided flexibility and scalability: those designed for a single algorithm that has some stochasticity, and those that are based on finding a "median consensus" of several clusterings. Here we describe these two types of methods.

The first type of method is designed to work with a single algorithm, such as Leiden or Louvain, that is non-deterministic due to its use of randomness, and so may return different outcomes in different runs. The first method of this type was developed in 2012 by Lancichinetti and Fortunato (2012). This method, referred to as "LF" in Hussain et al. (2025), runs Louvain (designed for modularity optimization) repeatedly. After a set number of applications of Louvain, the co-clustering matrix is computed, and all entries below some user-specified threshold are deleted. This algorithm iterates until the co-clustering matrix converges. While the LF-consensus method had good accuracy, the need for convergence made it computationally infeasible. Subsequent extensions, such as FastConsensus (Tandon et al. 2019), Ensemble Clustering for Graphs (ECG) (Poulin and Théberge 2020), and FastEnsemble (Tabatabaee et al. 2024), were developed to improve scalability and reduce runtime.

The second type of approach highlighted by Hussain et al. (2025) takes as input a collection of different clusterings of the same network, and seeks a clustering that minimizes the total distance to the input clusterings; this is analogous to computing the median under the L_1 norm of a set of points in a high-dimensional space. How the distance is defined affects the output both in terms of accuracy and runtime, but most approaches have used the Mirkin distance (Mirkin 2013), which is based on minimizing disagreements between clusterings, and that is NP-hard to solve exactly. The median consensus method introduced by Hussain et al. (2025) employs an effective greedy heuristic along with parallelism to achieve high scalability, and provides a substantial improvement over the LF-consensus for accuracy and scalability.

The ensemble technique from Vu-Le et al. (2026)

In Vu-Le et al. (2026), we developed and studied a method for combining two or more clusterings of a network. Here we describe the method we studied, and summarize the trends observed for this method.

The method we studied is very simple, and is based on allowing the input clusterings to vote on whether to keep an edge in the network, and if so what edge weight to assign. The standard "voting" approach would determine the proportion of clusterings that co-cluster the endpoints of the edge, and if that proportion is less than the user-specified threshold, then the edge is deleted. Any edge that is kept is then weighted by the proportion of clusterings that co-clustered the endpoints. This strategy produces an edge-weighted network called the "consensus network" that contains a (possibly proper) subset of the edges of the original network, which can then be clustered using a user-specified final clustering method.

We departed from this standard approach by only allowing a clustering method to vote on an edge (u, v) if both u and v appear in non-singleton clusters. Otherwise, that clustering method is not allowed to vote (i.e., it abstains), and so it does not get counted when calculating the proportion of input clusterings that voted for keeping the edge. For example, if the input has three clusterings but only two of them put both a and b into non-singleton clusters (where (a, b) is an edge in the network), then only those two clusterings are allowed to vote on whether to keep the edge (a, b). In other ways, the method we studied follows the same paradigm. The user specifies (1) the threshold t that determines if the edge is maintained or not, (2) the clusterings that are used to vote on edges, and (3) the final clustering method used to cluster the modified and weighted network. We also allowed the user to specify whether the edge weights are considered in the final clustering; if so, then the final clustering method must be able to handle edge-weighted networks.

In Vu-Le et al. (2026), we evaluated this technique when the input clusterings were DSC-Flow-Iter and Leiden-Mod, and the final clustering method was Leiden-CPM using resolution value 0.01 and then post-processed by WCC (we refer to this as Leiden-CPM(0.01)+WCC). The study explored different values for t and whether to use edge weights, and settled on $t = 0.5$ and unweighted edges as the best solution.

Materials and methods

Clustering methods

In this work, we study the five DSC techniques described in Background: DSC-FISTA(int), DSC-Flow, DSC-FISTA-Iter, DSC-FISTA(int)-Iter, and DSC-Flow-Iter. Along with the DSC techniques, we also study a variety of clustering methods, including Leiden-Mod, Leiden-CPM with multiple resolution values r (denoted by Leiden-CPM(r)), Infomap, IKC with multiple B values but without modularity filtering (denoted by IKC(B)), and RTRex. Since Infomap can produce internally disconnected clusters, we always post-process Infomap with the CC technique (see Background). We also explore the impact of post-processing using the WCC technique in Experiment 3.

New cluster ensemble techniques

In this study, we explore a class of ensemble methods that has the following algorithmic components:

1. The set of clusterings given as input, which are used to define the consensus network
2. A method for determining whether to keep an edge in the network, and if so, what weight to assign it
3. A final clustering method that is then applied to the consensus network (that may not contain all the edges of the input network).

For (1), we do not constrain the possible input clusterings, but note that some clustering methods may have low node coverage (i.e., many nodes are in singleton clusters). Similarly, for (3), we do not constrain the final clustering method, but note that only some methods are able to handle weighted edges.

For (2), we have a fairly simple rule based on voting, as we now describe. Consider an edge in the input network. Each clustering is a voter for this edge. There are two voting schemes. In the *free voting* scheme (named "Free"), every clustering is eligible to vote. On the other hand, in the *constrained voting* scheme (named "Constrained"), only clusterings that cluster both endpoints are eligible to vote. To assign a weight to the edge, the eligible votes are tallied. In the free voting scheme, the weight is the non-negative integer count. In the constrained voting scheme, the weight is the count divided by the number of eligible voters, which is a value between 0.0 and 1.0 (and if there are no eligible votes, we set the weight to 0.0).

There are four consensus variants that determine which edges we include, based on the votes that are received:

- *fully-weighted* (named "Full"): no edges are removed but are weighted, which means edges with no support are effectively removed.
- *majority rule* (named "Majority"): an edge is kept if it is supported by at least 50% of eligible voters.
- *strict majority rule* (named "StrictMajority"): an edge is kept if it is supported by more than 50% of eligible voters.
- *strict consensus* (named "Strict"): an edge is kept if all eligible voters support it.

This way of describing the ensemble method is slightly different from the one presented in Vu-Le et al. (2026), which used a user-specified threshold t to decide which edges to include. We also enable this in the code, but in this study, we only explore the above variants of the technique.

Recall that (Vu-Le et al. 2026) studied a version of this framework, specifically constrained voting with majority rule using two voters (DSC-Flow-Iter and Leiden-Mod) and then clustering the consensus network with Leiden-CPM(0.01) with WCC post-processing. Hence, here we examine a larger set of variants of this basic approach.

To motivate the constrained clustering approach, we note that constrained voting has some desirable theoretical properties. Specifically, if the input clusterings used to construct the consensus network have perfect precision (i.e., none of them put two nodes in the same cluster if they do not appear in the same ground truth cluster) and the final clustering method does not produce clusters that are internally disconnected, then using constrained voting will also have perfect precision (Theorem 1 in the Supplementary Materials). Moreover, if in addition the following properties hold: (1) the threshold for keeping an edge is met by having at least one clustering putting the endpoints in the same cluster and (2) if every pair of nodes that are in the same ground truth cluster are in some input cluster, then the components of the consensus network will be the ground

truth clusters (Theorem 2 in the Supplementary Materials). These theoretical guarantees have not been established for prior methods, and we would conjecture they will fail for the Median Consensus and the Free Voting Consensus. Nevertheless, because perfect precision cannot be expected in any clustering method, in the experiments below, we explore different rules that allow for imperfect precision (e.g., majority consensus and strict consensus).

Study design

To design and evaluate the pipelines, we use both synthetic and real-world networks.

Datasets

Real-world networks

We analyze 78 real-world networks taken from Peixoto (2020) and Park et al. (2024a). The collection encompasses a wide array of applications with vertex counts from 906 to nearly 14 million. The comprehensive catalog can be found in Supplementary Materials Section S1. To assess the accuracy of community detection methods, we used synthetic networks created based on 74 of these real-world networks, excluding the four largest ones (`livejournal`, `orkut`, `bitcoin`, and `CEN`). On the other hand, for evaluating scalability and computational runtime benchmark, the entire suite of 78 networks was used.

Synthetic networks

We used EC-SBM benchmark networks (Vu-Le et al. 2025a, b) for our analysis. EC-SBM derives generative parameters from a reference clustered network and subsequently synthesizes a counterpart that faithfully replicates the topological properties of the original network and its clusters. In our generation pipeline, we employed the 74 selected real-world networks as the reference networks. Their corresponding reference clusterings were computed via stochastic blockmodel inference using the graph-tool library, followed by WCC post-processing (detailed in the Background section). We denote this as the SBM+WCC clustering. Because this post-processing guarantees that every community in the reference graph is well-connected, EC-SBM ensures by design the same property for the synthetic communities. We adopted the SBM+WCC reference clustering motivated by empirical observations in Vu-Le et al. (2026), which demonstrated that this specific configuration yields the most accurate replication of the source clustered network's statistical properties.

Furthermore, we randomly split the corpus of synthetic networks into two parts. The *training dataset* has 51 networks while the *testing dataset* has 23 networks. In designing the ensemble method, we used the training dataset to determine the best algorithmic parameters. Then, the testing dataset was used to evaluate and compare the chosen method with other techniques. Supplementary Materials Section S1 also lists the networks comprised in each split.

Criteria

We use two sets of criteria. The first set contains three *clustering criteria*: Adjusted Mutual Information (AMI) (Vinh et al. 2009), Adjusted Rand Index (ARI) (Hubert and Arabie 1985), and Normalized Mutual Information (NMI) (Forbes 1995). However, we pay less attention to NMI accuracy than ARI and AMI, since NMI is known to be biased

towards clusterings that have many clusters. For the second set, we reframe the clustering problem as a classification problem: for each pair of nodes, we ask if they belong to the same cluster (each unclustered node is considered to be a singleton cluster). We can then consider three conventional *classification criteria*: F1 score, precision, and recall (note that F1 is the harmonic mean of precision and recall).

Experiments

We conducted five experiments: We use all 74 EC-SBM networks for Experiments 1 and 2, the training datasets (51 EC-SBM networks) for Experiment 3, and then the testing datasets (23 EC-SBM networks) for Experiment 4. For Experiment 5, we use 78 real-world networks.

- Experiment 1: We compare different density-based techniques and establish that DSC-Flow-Iter is the best of all the density-based techniques we studied. This experiment extends the results from Vu-Le et al. (2026) by also exploring an iterative version of DSC-FISTA(int).
- Experiment 2: We compare DSC-Flow-Iter with standard community detection methods, extending the results from Vu-Le et al. (2026) by including RTRex.
- Experiment 3: We design a new cluster ensemble using the training dataset. This experiment extends (Vu-Le et al. 2026) in two ways: we explore more variants of the basic algorithmic strategy used to combine clusterings, and we also explore how to select the set of community detection methods to put in the cluster ensemble.
- Experiment 4: We evaluate the new cluster ensemble technique using the testing dataset in comparison to other community detection methods.
- Experiment 5: We benchmark the runtime of the proposed pipeline on real-world networks. This experiment is based on a reimplementation of the earlier code for DSC-Flow-Iter studied in Vu-Le et al. (2026) to improve its memory usage.

All experiments were conducted on the Illinois Campus Cluster, using nodes with 256GB of RAM. The computation time for each task (running a community detection algorithm on a network, or running the post-processing treatment on a clustering result) was limited to 3 days.

Experimental results

In our experiments, we focus mainly on accuracy, using AMI, ARI, NMI, precision, recall, and F1.

Experiment 1: Comparing DSC methods

In this experiment, we compare methods for community detection that are based either on the FISTA or the Flow algorithms from Harb et al. (2022) and Harb et al. (2025), respectively. Recall that (Vu-Le et al. 2026) found that FISTA yields a granular, staircase-like output, making simple integer rounding (DSC-FISTA(int)) an effective heuristic. Conversely, Flow produces a coarser partition where a single broad plateau merges multiple FISTA steps. The analysis from Vu-Le et al. (2026) studied several methods in this category, but not the variant DSC-FISTA(int)-Iter, which applies the integer-rounding heuristic within the iterative framework in place of fractional peeling. Thus, in this study, we extend (Vu-Le et al. 2026) by including DSC-FISTA(int)-Iter.

Table 1 Comparison of DSC methods on synthetic networks with respect to clustering criteria

Method	AMI		ARI		NMI	
	Avg	# best	Avg	# best	Avg	# best
DSC-FISTA(int)	0.285	14	0.066	24	**0.849**	**50**
DSC-FISTA(int)-Iter	0.309	7	0.051	5	0.779	0
DSC-FISTA-Iter	0.123	1	0.010	1	0.359	0
DSC-Flow	0.265	6	0.041	3	0.725	1
DSC-Flow-Iter	**0.370**	**46**	**0.121**	**45**	0.833	23

The results are on 74 synthetic EC-SBM networks with SBM+WCC ground-truth clustering. DSC-Flow-Iter is the method with the highest average accuracy for AMI and ARI, and second best for NMI

Table 2 Comparison between DSC-Flow-Iter and prior community detection methods on 74 synthetic EC-SBM networks with SBM+WCC ground-truth clustering

Method	AMI		ARI		NMI		Node Coverage
	Avg	# best	Avg	# best	Avg	# best	Avg
DSC-Flow-Iter	0.370	17	0.121	15	0.833	6	0.635
RTRex	0.099	5	0.039	4	**0.907**	**52**	0.071
Leiden-CPM(0.01)	**0.434**	**38**	**0.241**	**40**	0.829	14	0.839
Leiden-Mod	0.272	2	0.073	4	0.504	0	1.000
Infomap	0.229	4	0.071	5	0.447	2	1.000
IKC(1)	0.307	8	0.060	6	0.802	0	0.607

As Table 1 illustrates, the new variant, DSC-FISTA(int)-Iter, achieves a higher average AMI (0.309) than both its non-iterative counterpart, DSC-FISTA(int) (0.285), and the fractional peeling approach, DSC-FISTA-Iter (0.123). This improvement likely stems from the fact that the iterative procedure only needs to identify the single densest subgraph at each step, which potentially has a better approximation than the following subgraphs. However, DSC-Flow-Iter still maintains the highest average AMI (0.370) and ARI (0.121), ranking best on 46 and 45 networks, respectively. DSC-Flow-Iter has lower NMI than DSC-FISTA(int) but has better NMI accuracy than the other methods. It therefore remains our primary DSC representative.

Experiment 2: Evaluating DSC-Flow-Iter

In Vu-Le et al. (2026), we benchmarked our top-performing DSC method, DSC-Flow-Iter, against other community detection algorithms: Leiden-CPM at various resolution parameter values, Leiden-Mod, Infomap, and IKC using various values for B. The previous study selected the parameter values to obtain better AMI and ARI scores, which resulted in $B = 1$ for IKC and resolution parameter 0.01 for Leiden-CPM (see Supplementary Materials Table S1). In this extended analysis, we include RTRex, a method based on triangle-density extraction.

After including RTRex, our findings are consistent with those in Vu-Le et al. (2026), but show how RTRex compares to the other methods. We find that Leiden-CPM(0.01) has better AMI and ARI than all other tested methods (see Table 2). RTRex had poor AMI and ARI (the worst of the tested methods) but the best NMI. We find that DSC-Flow-Iter is in second place for AMI and ARI, and second best for NMI.

We now consider why RTRex had a very high NMI score (best in 52 of the 74 tested networks), despite having the lowest scores on AMI (0.099) and ARI (0.039). Table 2 shows that RTRex placed close to 93% of the nodes into singleton clusters on average. Furthermore, low node coverage for RTRex has also been observed in Basu et al. (2024).

This exceptionally low node coverage explains why RTRex achieves a high NMI score, given the established tendency of NMI to favor clusterings with a large number of clusters (even singleton clusters).

Experiment 3: Designing the cluster ensemble method

In this experiment, we use the training dataset to design the cluster ensemble pipeline. Recall that our cluster ensemble method has three algorithmic components: the set of base clustering methods, the edge weighting configuration based on voting, and the final clustering algorithm. We performed three sub-experiments to make the choice for each component.

Experiment 3a: Selecting the final clustering algorithm

The goal of this sub-experiment is to determine the best community detection algorithm to use as the final clustering algorithm for our ensemble technique. Specifically, we compare DSC-Flow-Iter, RTRex, Leiden-CPM at various resolutions, Leiden-Mod, Infomap, and IKC at various B values, each with and without WCC postprocessing. We then select the method that has the best AMI and ARI accuracy.

Detailed results, provided in Supplementary Materials Table S2, demonstrate that Leiden-CPM(0.01)+WCC emerges as the dominant technique. It achieves the highest average AMI (0.465) and ARI (0.234), substantially outperforming the next strongest alternative, Leiden-Mod+WCC, which yields an AMI of 0.424 and a notably lower ARI of 0.126. Consequently, we designate Leiden-CPM(0.01)+WCC as the algorithm for clustering the consensus network in our ensemble architecture.

We also note that WCC tends to be beneficial for most methods. However, methods that produce clusters that are dense in some sense, like the iterative extraction techniques, tend to be less affected by WCC. This is most evident by the small change in accuracy produced by WCC post-processing for DSC-Flow-Iter, IKC with large values for B (e.g., $B = 5, 10$, or 20), and RTRex. Remarkably, WCC postprocessing also has a very small impact on Leiden-CPM(0.1). This trend is likely explained by the observation that these methods tend to produce dense clusters, and clusters that are very dense are often well-connected (i.e., a cluster that has many edges will typically not have a small edge cut).

Experiment 3b: Selecting the clustering methods to combine

To construct an effective cluster ensemble, it can be helpful to select a collection of relatively accurate but diverse clustering methods (Golalipour et al. 2021, 2024). Therefore, we sought base clustering algorithms that exhibit complementary accuracy profiles across classification metrics, specifically focusing on the trade-off between precision and recall. Results on the training data are provided in Supplementary Materials Table S3. Because Leiden-CPM(0.01)+WCC is utilized as the final consensus clustering method, we exclude Leiden-CPM with other resolution parameters from consideration as a base clustering to ensure diversity in the ensemble collection.

For our high-recall representative, we selected Leiden-Mod: it achieves a near-perfect average recall of 0.927, indicating that it successfully captures the vast majority of true co-clustered pairs, and had 100% node coverage. The only other method with 100% node coverage was Infomap, which had lower recall (0.878). Leiden-Mod's expansive coverage comes at the cost of low precision (0.034). To counterbalance modularity, we selected three precision-oriented methods: RTRex, DSC-Flow-Iter, and IKC(5). By strictly

focusing on triangle-dense communities, RTRex has the highest average precision in our suite (0.612), but has negligible recall (0.020), which explains its poor performance in overall clustering accuracy. DSC-Flow-Iter and IKC(5) bridge the gap between these extremes, providing decent precision (between 0.052 and 0.087) while maintaining moderate recall (0.374 and 0.201, respectively).

We note that the chosen methods differ in terms of node coverage. On the training datasets, Leiden-Mod has 100% node coverage followed by DSC-Flow-Iter with 61.7%, but IKC(5) and RTRex have node coverage lower than 20% (Supplementary Materials Table S3). Thus, by including some methods that have high precision but low node coverage with methods that have higher recall and node coverage but lower precision, we hope to produce a cluster ensemble containing diverse and complementary input partitions that we can then use to construct the consensus network.

Experiment 3c: Comparing different cluster ensemble variants

Based on results from Experiment 3b, we selected four base clusterings for the cluster ensemble: DSC-Flow-Iter (denoted by F), Leiden-Mod (denoted by M), RTRex (denoted by R), and IKC(5) (denoted by K), utilizing Leiden-CPM(0.01)+WCC (denoted by C) to cluster the resulting consensus network (see also Table 3 for the mapping of names to methods). In this experiment, we evaluate techniques for combining these clusterings using our basic cluster consensus method.

We structured the evaluation hierarchically. We started with a minimal ensemble of two base clusterings, DSC-Flow-Iter and Leiden-Mod. We then incorporated RTRex and subsequently IKC(5) to assess the impact on accuracy for including each method in the ensemble. The results are summarized in Table 4. In this table, the set of selected consensus methods, including Leiden-CPM(0.01)+WCC (denoted by C) is denoted

Table 3 Glossary of clustering method and pipeline names

Name	Meaning
F	DSC-**F**low-Iter
M	Leiden-**M**od
R	**R**TRex
K	I**K**C(5)
C	Leiden-**C**PM(0.01)+WCC
CVC	**C**onstrained **V**oting **C**onsensus
FVC	**F**ree **V**oting **C**onsensus
MedCon	**Med**ian **Con**sensus
FMRKC-CVC	Input clusterings: F, M, R, K Final clustering: C Default: CVC is used with majority rule producing an unweighted network Options: Full (all eligible votes), Strict Majority, Strict Consensus, and the network is edge-weighted. Note: FMC-CVC represents the case where the input clusterings are F and M; FMRC-CVC represents the case where the input clusterings are F, M, and R.
FMRKC-FVC	Input clusterings: F, M, R, K Final clustering: C Default: FVC is used with majority rule producing an unweighted network Options: Full (all eligible votes), Strict Majority, Strict Consensus, and network is edge-weighted Note: FMC-FVC represents the case where the input clusterings are F and M; FMRC-FVC represents the case where the input clusterings are F, M, and R.
FMRKC-MedCon	Input clusterings: F, M, R, K, and C MedCon is used
MC-MedCon	Input clusterings: M and C. MedCon is used

Table 4 Comparisons of different ensemble methods with respect to clustering criteria on training dataset

Method	AMI		ARI		NMI	
	Avg	# best	Avg	# best	Avg	# best
FMC-Constrained-Full	0.465	0	0.278	1	0.881	0
FMC-Free-Full	0.484	1	0.277	1	0.879	0
FMC-Constrained-Majority (W)	0.467	0	0.281	0	0.882	0
FMC-Free-Majority (W)	0.481	1	0.273	0	0.878	0
FMC-Constrained-Majority (U)	0.484	1	0.278	1	0.883	1
FMC-Free-Majority (U)	0.484	0	0.278	1	0.883	1
FMC-Constrained-Strict	0.447	1	0.268	1	0.893	0
FMC-Free-Strict	0.443	2	0.232	2	0.908	4
FMRC-Constrained-Full	0.466	2	0.278	2	0.881	1
FMRC-Free-Full	0.484	3	0.276	1	0.879	0
FMRC-Constrained-Majority (W)	0.468	0	0.283	1	0.882	0
FMRC-Free-Majority (W)	0.438	3	0.223	3	0.901	2
FMRC-Constrained-Majority (U)	0.483	6	0.278	2	0.882	5
FMRC-Free-Majority (U)	0.442	3	0.228	3	0.907	1
FMRC-Constrained-Strict	0.443	0	0.261	0	0.892	0
FMRC-Free-Strict	0.104	2	0.046	2	**0.916**	**21**
FMRKC-Constrained-Full	0.468	0	0.287	2	0.883	0
FMRKC-Free-Full	0.480	6	0.263	4	0.878	1
FMRKC-Constrained-Majority (W)	0.468	2	0.295	3	0.886	1
FMRKC-Free-Majority (W)	0.429	0	0.200	1	0.890	0
FMRKC-Constrained-Majority (U)	**0.486**	**7**	**0.301**	**11**	0.885	5
FMRKC-Free-Majority (U)	0.435	1	0.214	1	0.897	0
FMRKC-constrained-StrictMajority (W)	0.444	0	0.261	0	0.888	0
FMRKC-Free-StrictMajority (W)	0.291	1	0.126	4	0.908	1
FMRKC-Constrained-StrictMajority(U)	0.444	0	0.256	0	0.887	0
FMRKC-Free-StrictMajority(U)	0.291	5	0.131	2	0.911	5
FMRKC-Constrained-Strict	0.440	3	0.273	9	0.894	2
FMRKC-Free-Strict	0.064	2	0.039	2	0.913	12

See Table 3 for translation of ensemble method names. For each method and criterion, the table shows the average value attained by the method across all networks and the number of networks on which the method produces the largest value. In case of ties, all networks with the best value are counted, so the sum of counts may exceed 51.

by a string of letters ending in "C". Thus, "FMC" indicates that the ensemble includes F (DSC-Flow-Iter) and M (Leiden-Mod), and these are then combined with C (Leiden-CPM(0.01)+WCC). Please see Table 3 for the mapping of these strings to pipelines.

We focus on AMI and ARI, since NMI is known to be biased towards clusterings that have many clusters (Mahmoudi and Jemielniak 2024).

For a given ensemble (i.e., a collection of clusterings), the best technique for combining the clusterings (e.g., whether to use constrained or free voting, full or majority vote, and weighted or unweighted edges) depends on the members in the ensemble as well as on the accuracy criterion (Table 4). However, the best overall results for AMI and ARI are obtained using all four clusterings in the ensemble (i.e., FMRK) and then combining them with Constrained Voting, Majority rule, and unweighted edges (i.e., FMRKC-Constrained-Majority(U), see Table 3). The FMRKC-Constrained-Majority(U) configuration performs best across the experimental suite for average AMI and ARI scores. It ties for the highest average AMI (0.486) and has the uniquely best average ARI (0.301); it also has the best AMI and ARI scores on the largest number of networks in the training dataset.

Other general trends are worth noting. Under most conditions, constrained voting has better AMI and ARI accuracy than free voting, and the gap between them increases as the number of low node-coverage clusterings increases. This trend emphasizes the importance of constrained voting. Another clear (but expected) trend is that the Strict Consensus variants have lower AMI and ARI accuracy than Majority, especially when used with free voting instead of constrained voting. Again, unsurprisingly, the method with the best NMI score is consistently Free-Strict, which produces the most clusters since it removes the most edges. Another key finding is that as long as constrained-majority voting is applied, incrementally incorporating more base clusterings is either neutral or beneficial for AMI and ARI accuracy.

Based on these observations, we propose the protocol that had the best AMI and ARI accuracy, denoted in the table by FMRKC-Constrained-Majority(U). Given the importance of constrained voting, we refer to this more simply as FMRKC-CVC, so that FMRKC defines the input clusterings (FMRK) and final clustering (C), and CVC refers to Constrained Voting Consensus. We summarize this protocol explicitly as follows:

1. Stage 1: Run DSC-Flow-Iter on the input network
2. Stage 2: Run Leiden-Mod, RTRex, and IKC(5) on the input network
3. Stage 3: Construct an unweighted consensus network using the constrained voting strategy at the majority rule consensus level
4. Stage 4: Run Leiden-CPM(0.01) on the consensus network and apply WCC post-processing to the resulting clustering

In the following experiment, we also compare the protocol with its free voting variant, which will be called FMRKC-FVC (FVC stands for Free Voting Consensus).

Experiment 4: Evaluating the FMRKC-CVC cluster ensemble

We evaluate the accuracy of our proposed cluster ensemble pipeline (FMRKC-CVC) on the testing dataset in comparison to an alternative variant of our ensemble approach with Free Voting (FVC), the Median Consensus from Hussain et al. (2025), as well as to its constituent methods. We also explore the impact of removing one or more clustering methods from the ensemble for these clustering consensus methods. Complete results are in Supplementary Materials Tables S4 and S5; here, we highlight the interesting trends.

Comparison to Leiden-CPM(0.01)+WCC. We begin with a comparison between our proposed pipeline with the most accurate of its constituent clustering methods, which is Leiden-CPM(0.01)+WCC (see Table 5 and Supplementary Materials Tables S4 and S5). Our ensemble is more accurate than Leiden-CPM(0.01)+WCC across all clustering criteria, achieving higher average scores for AMI (0.625 vs. 0.583), ARI (0.439 vs. 0.367), and NMI (0.906 vs. 0.888). Furthermore, for all clustering criteria, the ensemble is more accurate on the vast majority of individual networks, securing the best result on

Table 5 Comparison of FMRKC-CVC to Leiden-CPM(0.01)+WCC with respect to clustering criteria on the testing dataset.

Method	AMI		ARI		NMI	
	Avg	# best	Avg	# best	Avg	# best
Leiden-CPM(0.01)+WCC	0.583	3	0.367	3	0.888	3
FMRKC-CVC	**0.625**	**20**	**0.439**	**20**	**0.906**	**20**

20 out of the 23 test cases. Results for F1, precision, and recall show similar trends (Supplementary Materials Table S5), and the difference in F1 score is statistically significant (p=0.0010 under the Benjamini-Hochberg adjustment and under the Holm-Bonferroni adjustment, Tables S8 and S9 in the Supplementary Materials).

By design, the difference between the ensemble and Leiden-CPM(0.01)+WCC is essentially only in the input network to the clustering algorithm, specifically that the latter takes the original input network, while the former takes the consensus network where edges are removed by considering votes from single clustering algorithms. The result shows that this removal of edges through combination of multiple single algorithms helps lead to better community detection.

Comparison to Free Voting and Median Consensus We then compare our CVC ensemble pipeline to free voting (FVC) and the Median Consensus method, given the same set of clustering methods (i.e., FMRKC-MedCon). Figure 1 and Supplementary Materials Table S4 show results for AMI, ARI, NMI and Supplementary Materials Table S5 shows results for F1, precision, and recall. These show that our constrained voting approach (FMRKC-CVC) outperforms the free voting variant (FMRKC-FVC), illustrating the critical role of filtering out ineligible voters (base clusterings that do not put an endpoint into any cluster) in the edge-weighting process.

The CVC pipeline (FMRKC-CVC) is also more accurate than the Median Consensus approach (FMRKC-MedCon) in terms of average values for all criteria (Figure 1 and Supplementary Materials Table S4 show results for AMI, ARI, NMI and Supplementary Materials Table S5 shows results for F1, precision, and recall). Furthermore, the difference in F1 score is statistically significant (p=0.0070 usng the Benjamini-Hochberg adjustment and p=0.0105 under the Holm-Bonferroni adjustment, see Supplementary Materials Section S6 Tables S8 and S9).

Ablation study for CVC, FVC, and Median Consensus

We then performed an ablation study where we removed input clustering methods, thus producing an input of fewer clusterings, to determine the impact on accuracy. We were particularly interested in the impact of removing clusterings with high node coverage compared to removing clusterings with very low node coverage. We also observed that the base clusterings exhibit distinct characteristics (Figure 2) that we also observed on the training dataset: Leiden-Mod achieves high recall (0.929) and high node coverage

Method	AMI		ARI		NMI	
	Avg	# best	Avg	# best	Avg	# best
FMRKC-CVC	**0.625**	**17**	**0.439**	**14**	**0.906**	**11**
FMRKC-FVC	0.531	4	0.312	5	0.895	3
FMRKC-MedCon	0.517	2	0.274	4	0.901	9

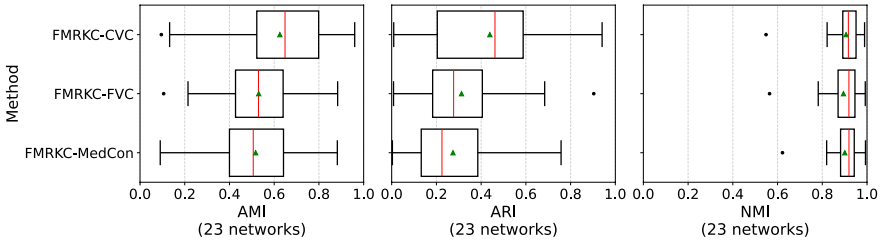


Fig. 1 Comparisons of three ensemble methods with respect to clustering criteria on the testing dataset. The results are on 23 synthetic EC-SBM networks with SBM+WCC ground-truth clustering (testing dataset). (See Table 3 for mapping names to methods)

Method	F1		Precision		Recall	
	Avg	# best	Avg	# best	Avg	# best
RTRex	0.024	0	**0.655**	**17**	0.013	0
Leiden-Mod	0.121	1	0.077	0	**0.929**	**23**
FMRKC-CVC	**0.443**	**15**	0.421	1	0.599	0
FMRKC-MedCon	0.277	7	0.400	5	0.270	0

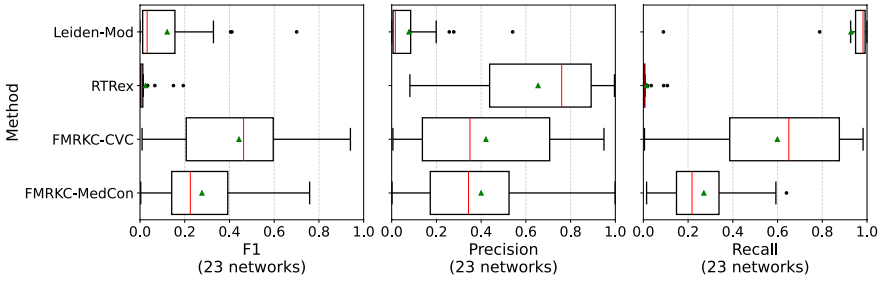


Fig. 2 Comparisons of ensemble methods and their constituent methods with respect to classification criteria on the testing dataset Our ensembles use four base clusterings (see Table 3 for mapping names to methods)

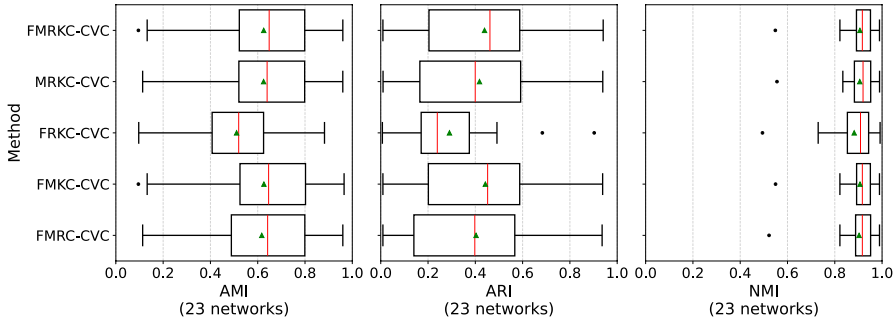


Fig. 3 Ablation study for the proposed protocol with respect to clustering criteria The results are on 23 synthetic EC-SBM networks with SBM+WCC ground-truth clustering (test dataset). (See Table 3 for mapping names to methods)

but very low precision (0.077), whereas RTRex is more precise (0.655) but suffers from negligible recall (0.013) and very low node coverage.

Beginning with the CVC pipeline (Figure 3 and Supplementary Materials Figure S1), we see that removing Leiden-Mod from the input clusterings reduces AMI and ARI accuracy substantially; this is expected, since the node coverage for the other methods is very low, and together they cover only a small portion of the network. In fact, removing Leiden-Mod even reduces NMI, but not by as large an extent. This emphasizes the need for some input clustering (or set of clusterings) to provide high node coverage and recall. Removing DSC-Flow-Iter does not change AMI but does reduce ARI accuracy, while removing IKC(5) reduces both AMI and ARI, so that it is best to keep both in the input. Finally, removing RTRex seems to have very little impact. Supplementary Materials Figure S2 shows that the same trends hold in terms of precision, recall, and F1 score. These results show that the CVC protocol is highly robust to the inclusion of low coverage clusterings that have good precision, but need some high node coverage clusterings to obtain good accuracy.

We next examined the impact of deleting clustering methods from the input given to the Free Voting Consensus (FVC) method. Using the FVC pipeline with majority vote on the training data, we see (Table 4) that when we exclude the two methods with very

low node coverage (i.e., IKC(5) and RTRex) then accuracy improves, showing that FVC is impacted by including low node coverage clusterings. However, even when restricted to just the other clusterings, it is still not as accurate as our CVC pipeline with all the clustering methods.

We see a similar impact for the Median Consensus. Furthermore, its best F1 accuracy is achieved if its input is restricted to the two methods with high node coverage, i.e., only Leiden-Mod and Leiden-CPM(0.01)+WCC (Figure S4, Supplementary Materials). Moreover, given only those two clusterings, it achieves nearly the same F1 score as the CVC pipeline given all five clusterings (Figure 4), and the difference is not statistically significant (see Supplementary Materials Section S6, Tables S8 and S9). Results for ARI, AMI, and NMI show similar trends (Supplementary Materials Figure S3).

Overall, this ablation study shows that while the CVC ensemble method requires some input clusterings to have high node coverage and recall, the impact of including clustering methods that have low node coverage but high precision ranges from neutral (RTRex) to beneficial (DSC-Flow-Iter and IKC(5)). This distinguishes it from the FVC (Free Voting Consensus) method and the Median Consensus method, each of which has reduced accuracy when low node coverage clusterings are included.

Evaluating the impact of WCC Recall that the CVC pipeline is designed so that the final output clustering has been post-processed by WCC, which guarantees that every cluster is well-connected, but this guarantee does not hold for the Median Consensus. We therefore added an experiment where we post-process the Median Consensus clustering with WCC, so that both protocols have the same guarantee of meeting the well-connectedness threshold for all output clusters. The results for this experiment are provided in the supplementary materials (Tables S6 and S7), and show that applying WCC is neutral when the Median Consensus is given either all five input clustering methods or just the two methods (Leiden-Mod and Leiden-CPM(0.01)+WCC) that have good node coverage.

Experiment 5: Runtime for the pipeline

We evaluated the runtime of our proposed pipeline on 78 real-world networks. In Vu-Le et al. (2026), the implementation of Stage 1 (DSC-Flow-Iter) had memory issues that prevented us from completing analyses using the pipeline on five large networks (`google_web`, `livejournal`, `orkut`, `bitcoin`, and `CEN`). We subsequently diagnosed the

Method	F1		Precision		Recall	
	Avg	# best	Avg	# best	Avg	# best
FMRKC-CVC	**0.443**	**9**	**0.421**	6	**0.599**	**16**
MC-MedCon	0.433	8	0.420	4	0.557	6
FMRKC-MedCon	0.277	6	0.400	**13**	0.270	1

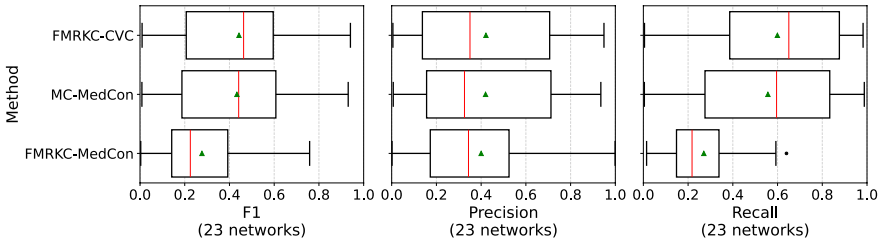


Fig. 4 Comparisons of ensemble methods with respect to classification criteria on the testing dataset. See Table 3 for mapping names to methods

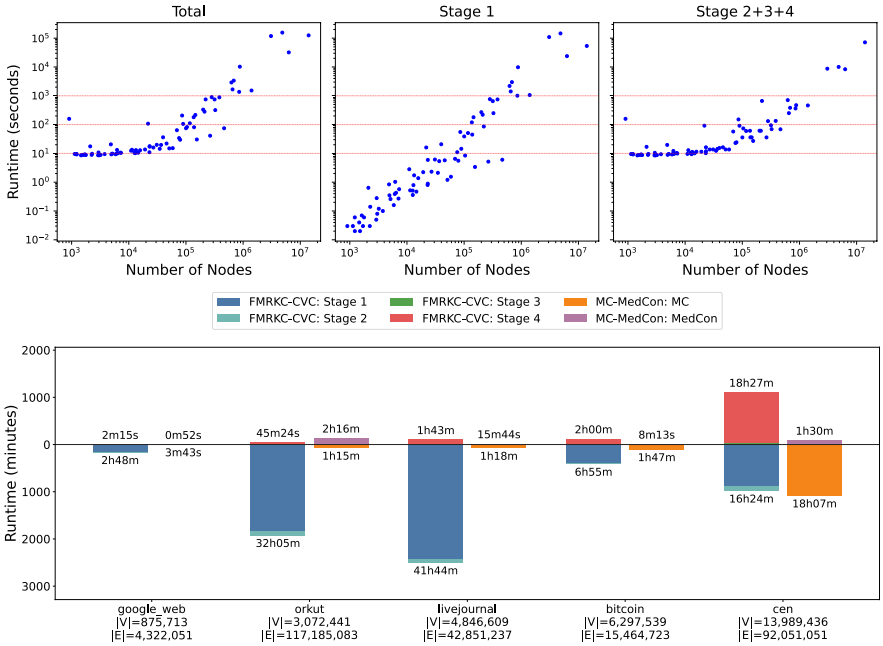


Fig. 5 Runtime of the pipeline on real-world networks. Top: scatterplot of runtimes (both *x*-axis and *y*-axis are in log-scale) on all 78 real-world networks. Bottom: runtimes by stage of the six largest real-world networks with 875,713 to 13,989,436 nodes. Stage 1 runs DSC-Flow-Iter; Stage 2 runs Leiden-Mod, RTRex, and IKC(5); Stage 3 uses the clusterings from the previous two stages to generate the unweighted network; and Stage 4 applies Leiden-CPM(0.01)+WCC to the network from Stage 3. The horizontal line splits the base clusterings generation stages (bottom) and the ensemble stages (top)

issue as a memory leak, resulting from repeated invocation of an external function in the iterative process. We resolved the issue and have completed the analyses on all 78 real-world networks.

We show the runtimes on the 78 real-world networks in our study, which range from fewer than 1,000 to nearly 14,000,000 nodes (`CEN`). Figure 5 (top) presents these runtimes in three panels; the left panel shows the total runtime, the middle panel shows Stage 1, which is dedicated to computing the DSC-Flow-Iter clustering, and the right panel shows the remaining steps. On the smaller networks (i.e., with at most 10^5 nodes), the total runtime is generally under a minute, but then increases as the number of nodes increases.

The trend is true for Stages 2–4, and especially for Stage 1, which shows a much faster increase. Specifically, Stage 1 exhibits a power-law relationship with the network size (appearing as a linear trend on the log-log scale in the figure). Overall, while the pipeline is very fast for the networks with under 10^5 nodes, it reached nearly 2608 minutes (43.5 hours) on the most computationally intensive network, `livejournal`.

We examined the runtime of the pipeline on the six largest real-world networks, which range from 875,713 nodes to 13,989,436 nodes. These results, shown in Figure 5 (bottom) reveal the following trends. First, while runtime tends to increase with the number of nodes, the dataset that took the most time was `livejournal`, which is not the largest (neither for the number of nodes nor the number of edges). Second, with the exception of the `CEN` network, the most computationally intensive step was always Stage 1 (i.e., computing the DSC-Flow-Iter clustering), which took the vast majority of the time on the other networks. On the three largest networks, Stage 4 (running

Leiden-CPM(0.01)+WCC on the resultant network) took a non-negligible portion of the runtime, and used more than 50% of the time for the largest network. Further investigation shows that the WCC step in the analysis is the computationally intensive part, as running Leiden-CPM(0.01) without WCC postprocessing is very fast.

These results, together, show that Stage 1 (i.e., DSC-Flow-Iter) is a major component of the runtime on the large networks and that the WCC part of Stage 4 is also computationally intensive for some networks.

Conclusions

In this study, we evaluated novel approaches for community detection, focusing on dense subgraph extraction and cluster ensembles. Building on an approach we introduced in Vu-Le et al. (2026), our consensus clustering protocol has three algorithmic components: (1) the choice of community detection methods to define the consensus network (containing only some of the edges of the input network), (2) the rule that determines which community detection methods can vote on whether to keep or delete an edge in the input network, and (3) a final clustering method to cluster the consensus network. We showed on synthetic networks that this protocol had high accuracy, improving on its constituent clustering methods and also providing better accuracy than the Median Consensus clustering. We improved the implementation from Vu-Le et al. (2026) to fix a critical memory leakage issue. With this optimization, the pipeline can now successfully scale to real-world networks comprising up to 14 million nodes, an achievement that was not possible using our initial implementation studied in Vu-Le et al. (2026).

Although there are similarities in this protocol to prior methods, no prior protocol had this structure. For example, the Median Consensus clearly allows any set of clustering methods to be combined, but it does not construct a consensus network and then cluster it. Many methods construct a consensus network (e.g., FastConsensus, FastEnsemble, ECG, etc.) but they each used a single clustering method (run multiple times to produce different clusterings) to create the consensus network and to finally cluster it, and they did not consider node coverage in constructing the consensus network. Thus, what distinguishes our protocol from prior protocols is not only the use of CVC for the rule specifying which methods can vote on keeping or deleting an edge, but also that we are combining techniques from prior clustering methods in a novel way. Finally, our protocol performed a careful selection of clustering methods to put into the ensemble; this is methodologically important.

These innovative aspects are also key to the high accuracy of the clustering ensemble method. Because the voting is constrained – only those clusterings that put both endpoints of an edge into clusters are allowed to vote on whether to keep the edge – this allows the protocol to benefit from the inclusion of clusterings that have very low coverage, provided that they have good precision. Because we can select which clustering methods are used to construct the consensus network and which one is used to finally cluster it, we were able to select methods with complementary properties for the consensus network construction (including ones that have very low node coverage but high precision), as well as one with high node coverage and good accuracy for the final step. This improves flexibility, robustness, and final accuracy.

In contrast, our evaluation of the Median Consensus showed that its accuracy degraded substantially when the ensemble includes low node coverage clustering

methods, even if they have high precision; this is in contrast to CVC, which is robust to including such clusterings. However, the Median Consensus does achieve high accuracy, comparable to the CVC pipeline using all five input clustering methods, if its input is restricted to just the two clustering methods with high node coverage.

The main limitation of the current approach is its runtime. Despite the current achieved scalability to networks with up to $\sim$ 14 million nodes, some analyses used nearly 44 hours of runtime, making the protocol potentially prohibitively expensive. Nevertheless, the CVC step itself is very fast, and our experiments identified DSC-Flow-Iter and the final WCC post-processing step as the primary computational bottlenecks. Improving the implementation of these methods to reduce the runtime, or replacing these methods with other approaches (e.g., replacing WCC by the Connectivity Modifier Park et al. 2024b), could improve the overall runtime of the protocol and might not reduce accuracy.

We leave several directions for future work. First, to address the runtime issues, we will develop modifications to DSC-Flow-Iter and WCC to improve their computational performance without reducing accuracy. Second, we will explore ways of improving accuracy for this protocol, such as techniques for weighted versions of constrained voting that are data-driven and take clustering properties into account. We will also examine how choices for the numeric parameters (e.g., the resolution parameter for Leiden-CPM or Leiden-Mod, the stopping rule for IKC, the expansion parameter for RTRex, etc) might affect accuracy. Given the flexibility and robustness of the CVC technique, there is substantial potential for even greater performance gains in future adaptations.

Supplementary Information

The online version contains supplementary material available at https://doi.org/10.1007/s41109-026-00809-z.

Supplementary file 1.

Author contributions

TAVL: designed study, implemented DSC techniques; performed experiments, analyzed data, wrote initial draft. MP: implemented the new cluster consensus method; performed experiments. IC: performed experiments. JACL: performed experiments. TA: performed experiments. EH: assisted in the design of DSC-based methods. GC: designed study, analyzed data. TW: designed study, analyzed data, wrote initial draft.

Funding

This work was supported in part by the Illinois-Insper partnership and by NSF grant 2402559 to TW and GC.

Data availability

We use the EC-SBM benchmark networks publicly available at Vu-Le et al. (2025b). The CVC software is available at the GitHub repository [40]. The DSC methods (e.g., DSC-flow-iter and other variants we studied) are available at the GitHub repository (Vu-Le 2025). The scripts for other methods and the commands run are detailed in Supplementary Materials Section S2.

Declarations

Competing interests

The authors declare that they have no competing interests.

Received: 22 February 2026 / Accepted: 1 June 2026

Published online: 09 June 2026

References

Basu S, Paul-Pena D, Qian K, Seshadhri C, Huang EW, Subbian K (2024) Covering a graph with dense subgraph families, via triangle-rich sets. Proceedings of the 33rd ACM International Conference on Information and Knowledge Management. pp 109–119

Blondel VD, Guillaume J-L, Lambiotte R, Lefebvre E (2008) Fast unfolding of communities in large networks. J Stat Mech: Theory Exp 2008(10):10008
Boob D, Gao Y, Peng R, Sawlani S, Tsourakakis C, Wang D, Wang J (2020) Flowless: Extracting densest subgraphs without flow computations. In: Proceedings of The web conference 2020. WWW '20, pp. 573–583. Association for Computing Machinery, New York, NY, USA. https://doi.org/10.1145/3366423.3380140
Charikar M (2000) Greedy approximation algorithms for finding dense components in a graph. In: Jansen K, Khuller S (eds) Approximation algorithms for combinatorial optimization. Springer, Berlin, Heidelberg, pp 84–95
Danisch M, Chan T-HH, Sozio M (2017) Large scale density-friendly graph decomposition via convex programming. In: Proceedings of the 26th international conference on world wide web. WWW '17, pp 233–242. International World Wide Web Conferences Steering Committee, Geneva, Switzerland. https://doi.org/10.1145/3038912.3052619
Forbes AD (1995) Classification-algorithm evaluation: five performance measures based on confusion matrices. J Clin Monit 11(3):189–206
Fortunato S, Barthélemy M (2007) Resolution limit in community detection. Proc Natl Acad Sci 104(1):36–41
Gallo G, Grigoriadis MD, Tarjan RE (1989) A fast parametric maximum flow algorithm and applications. SIAM J Comput 18(1):30–55. https://doi.org/10.1137/0218003
Golalipour K, Akbari E, Hamidi SS, Lee M, Enayatifar R (2021) From clustering to clustering ensemble selection: a review. Eng Appl Artif Intell 104:104388
Golalipour K, Akbari E, Motameni H (2024) Cluster ensemble selection based on maximum quality-maximum diversity. Eng Appl Artif Intell 131:107873
Goldberg AV (1984) Finding a Maximum density subgraph. University of California at Berkeley, USA. https://www2.eecs.berkeley.edu/Pubs/TechRpts/1984/CSD-84-171.pdf
Harb E, Quanrud K, Chekuri C (2022) Faster and scalable algorithms for densest subgraph and decomposition. Adv Neural Inf Process Syst 35:26966–26979
Harb E, Yassin Y, Chekuri C (2025) Corporate Needs You to Find the Difference: Revisiting Submodular and Supermodular Ratio Optimization Problems. arXiv:2505.17443
Hubert L, Arabie P (1985) Comparing partitions. J Classif 2:193–218
Hussain MT, Halappanavar M, Chatterjee S, Radicchi F, Fortunato S, Azad A (2025) Parallel median consensus clustering in complex networks. Sci Rep 15(1):3788
Lancichinetti A, Fortunato S (2012) Consensus clustering in complex networks. Sci Rep 2(1):336
Leskovec J, Lang KJ, Mahoney M (2010) Empirical comparison of algorithms for network community detection. Proceedings of the 19th international conference on world wide web. pp 631–640
Mahmoudi A, Jemielniak D (2024) Proof of biased behavior of normalized mutual information. Sci Rep 14(1):9021
Mirkin B (2013) Mathematical classification and clustering, vol 11. Springer, New York, NY
Newman ME (2004) Detecting community structure in networks. Eur Phys J B 38:321–330
Park M, Tabatabaee Y, Ramavarapu V, Liu B, Pailodi VK, Ramachandran R, Korobskiy D, Ayres F, Chacko G, Warnow T (2024) Well-connectedness and community detection. PLOS Complex Syst 1(3):1–25
Park M, Tabatabaee Y, Ramavarapu V, Liu B, Pailodi VK, Ramachandran R, Korobskiy D, Ayres F, Chacko G, Warnow T (2024) Well-connectedness and community detection. PLOS Complex Syst 1(3):1–25. https://doi.org/10.1371/journal.pcsy.0000009
Peixoto TP (2014) The graph-tool python library. figshare, http://figshare.com/articles/graph_tool/1164194
Peixoto TP (2020) The Netzschleuder network catalogue and repository. https://networks.skewed.de
Poulin V, Théberge F (2020) Comparing graph clusterings:set partition measures vs. graph-aware measures. IEEE Trans Pattern Anal Mach Intell 43(6):2127–2132
Rosvall M, Axelsson D, Bergstrom CT (2009) The map equation. Eur Phys J Spec Top 178(1):13–23
Tabatabaee Y, Wedell E, Park M, Warnow T (2024) FastEnsemble: A new scalable ensemble clustering method. In: International conference on complex networks and their applications. Springer, pp 57–70
Tandon A, Albeshri A, Thayananthan V, Alhalabi W, Fortunato S (2019) Fast consensus clustering in complex networks. Phys Rev E 99(4):042301
Traag V (2019) leidenalg. https://github.com/vtraag/leidenalg
Traag VA, Van Dooren P, Nesterov Y (2011) Narrow scope for resolution-limit-free community detection. Phys Rev E 84(1):016114
Traag VA, Waltman L, Van Eck NJ (2019) From louvain to leiden: guaranteeing well-connected communities. Sci Rep 9(1):1–12
Traag VA, Waltman L, Eck NJ (2019) From louvain to leiden: guaranteeing well-connected communities. Sci Rep 9(1):5233. https://doi.org/10.1038/s41598-019-41695-z
Vinh NX, Epps J, Bailey J (2009) Information theoretic measures for clusterings comparison: is a correction for chance necessary? In: Proceedings of the 26th annual international conference on machine learning. pp 1073–1080
Vu-Le T-A (2025) Dense Subgraph Clustering. https://github.com/illinois-or-research-analytics/DSC
Vu-Le T-A, Chacko G, Warnow T (2025) EC-SBM Benchmark Networks. University of Illinois Urbana-Champaign. https://doi.org/10.13012/B2IDB-3284069_V1
Vu-Le T-A. CVC. https://github.com/vltanh/CVC
Vu-Le T-A, Anne L, Chacko G, Warnow T (2025) EC-SBM synthetic network generator. Appl Net Sci 10(1):15
Vu-Le T-A, Park M, Chen I, Warnow T (2026) Using stochastic block models for community detection. Appl Netw Sci 11(1):2
Vu-Le T-A, Lamy JAC, Alessi T, Chen I, Park M, Harb E, Chacko G, Warnow T (2026) Dense subgraph clustering and a new cluster ensemble method. In: Cherifi H, Rocha LM, Cherifi C, Ertem Z (eds) Complex networks & their applications XIV. Springer, Cham, pp 29–40
Wedell E, Park M, Korobskiy D, Warnow T, Chacko G (2022) Center-periphery structure in research communities. Quant sci stud 3(1):289–314
Yang J, Leskovec J (2013) Defining and evaluating network communities based on ground-truth. Knowl Inf Syst 42(1):181–213

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.